

Pack 21 (Desktop) — Unattended Access Pre-Authorization Grants + Auto-Accept Guards (real code)

RemoteDesk · Pack 21 (Desktop) — Unattended Access Pre-Authorization Grants + Auto-Accept Guards (real code)

Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 → 20. Trust is now established (Pack 20), enforced (15), consented (16), funneled (17), reconnect-safe (18/19). This slice builds the one remaining policy field nothing has used yet: `unattendedAccessEnabled`. It adds time-boxed, scope-limited unattended-access grants so a session can start WITHOUT a human at the host, but only on a trusted device, within an explicit grant, and with auto-accept that still fails closed. Build order: shared grant contract → grant store → auto-accept evaluator → session-start wiring → UI → tests.

Scope: let a pre-authorized, trusted device start an unattended session under a revocable, expiring grant, while keeping every Pack 15→20 gate intact. **Fail-closed: no grant / expired / untrusted / blocked => unattended denied, falls back to normal consent. Grants are revocable and time-boxed. Host stays source of truth. No native executor.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]		
packages/shared/src/permissions/unattendedGrant.ts		
packages/shared/src/permissions/autoAccept.ts		
apps/desktop/src/renderer/src/services/unattendedGrantStore.ts		
apps/desktop/src/renderer/src/services/autoAcceptController.ts		
apps/desktop/src/renderer/src/components/UnattendedGrantPanel.tsx		
apps/desktop/src/renderer/src/components/UnattendedBanner.tsx		
tests/permissions/unattendedGrant.test.ts		
tests/permissions/autoAccept.test.ts		
tests/permissions/unattendedGrantStore.test.ts		
MODIFIED [MODIFY]		
packages/shared/src/permissions/index.ts	(export unattendedGrant + autoAccept)	
apps/desktop/src/renderer/src/services/sessionApprovalState.ts	(auto-approve only under a valid grant)	
apps/desktop/src/renderer/src/components/RemoteSessionView.tsx	(grant panel + unattended banner)	
NOT TOUCHED (intentionally)		
apps/desktop/src/main/index.ts	(no native executor)	
apps/desktop/src/preload/index.ts	(no new bridge)	
apps/desktop/src/renderer/src/services/webrtc.ts	(Pack 15/17 gates unchanged)	
apps/desktop/src/renderer/src/services/permissionState.ts	(Pack 15/16 unchanged)	
apps/api/** apps/web/**	(grant persistence assumed server-side; store falls back safely)	

Part A — Shared: unattended grant + auto-accept contracts

FILE: packages/shared/src/permissions/unattendedGrant.ts

```
import { deny, allow, type DeviceAccessPolicySnapshot, type PolicyDecision } from './policyEvaluator';
import type { FeatureKey } from './devicePolicy';
```

```
/**
 * A time-boxed, scope-limited pre-authorization that lets a trusted device start
 * an UNATTENDED session (no human at the host to consent). Grants are revocable
 * and expire. They never widen what the policy allows; they only substitute for
 * live host consent, and only for the features explicitly listed.
 */
```

```
export interface UnattendedGrant {
  grantId: string;
  deviceId: string;
  /** Which interactive features this grant pre-authorizes (subset of policy). */
  scope: FeatureKey[];
  /** ISO start; grant is not valid before this. */
  notBefore: string;
  /** ISO expiry; grant is invalid at/after this. */
  expiresAt: string;
  /** Set when an operator revokes early. */
  revokedAt: string | null;
  /** Non-sensitive label of who created it (display/audit only). */
  createdByLabel: string | null;
}
```

```
export type UnattendedDenialReason =
  | 'no_grant'
  | 'grant_revoked'
  | 'grant_not_yet_valid'
  | 'grant_expired'
  | 'device_not_trusted'
  | 'unattended_disabled_by_policy';
```

```
export interface UnattendedDecision {
  allowed: boolean;
  reason: UnattendedDenialReason | null;
  /** The effective scope the grant authorizes (empty when denied). */
  scope: FeatureKey[];
}
```

```
/** Is the grant valid at `now` (time window + not revoked)? Pure. */
export function isGrantActive(grant: UnattendedGrant, now: Date = new Date()): boolean {
  if (grant.revokedAt) return false;
  const t = now.getTime();
  return t >= Date.parse(grant.notBefore) && t < Date.parse(grant.expiresAt);
}
```

```
/**
 * Evaluate whether unattended access may proceed. Fail-closed and never broader
 * than the policy:
 * - policy must enable unattended access AND device must be trusted
 * - a non-revoked, in-window grant must exist
 * - effective scope = grant.scope intersected with policy-permitted features
 */
```

```

export function evaluateUnattendedAccess(input: {
  policy: DeviceAccessPolicySnapshot | null;
  grant: UnattendedGrant | null;
  now?: Date;
}): UnattendedDecision {
  const { policy, grant } = input;
  const now = input.now ?? new Date();

  if (!policy || policy.trustStatus === 'blocked' || policy.trustStatus === 'untrusted') {
    return { allowed: false, reason: 'device_not_trusted', scope: [] };
  }
  if (!policy.unattendedAccessEnabled) {
    return { allowed: false, reason: 'unattended_disabled_by_policy', scope: [] };
  }
  if (!grant) return { allowed: false, reason: 'no_grant', scope: [] };
  if (grant.revokedAt) return { allowed: false, reason: 'grant_revoked', scope: [] };
  if (now.getTime() < Date.parse(grant.notBefore)) {
    return { allowed: false, reason: 'grant_not_yet_valid', scope: [] };
  }
  if (now.getTime() >= Date.parse(grant.expiresAt)) {
    return { allowed: false, reason: 'grant_expired', scope: [] };
  }

  // Intersect grant scope with what the policy actually permits (never broader).
  const policyAllowed = policyPermittedFeatures(policy);
  const scope = grant.scope.filter((f) => policyAllowed.includes(f));
  return { allowed: true, reason: null, scope };
}

/** Features the policy itself permits (used to clamp grant scope). */
export function policyPermittedFeatures(policy: DeviceAccessPolicySnapshot): FeatureKey[] {
  const out: FeatureKey[] = [];
  if (policy.remoteInputEnabled) out.push('remoteInput' as FeatureKey);
  if (policy.clipboardSyncEnabled) out.push('clipboardSync' as FeatureKey);
  if (policy.fileTransferEnabled) out.push('fileTransfer' as FeatureKey);
  if (policy.unattendedAccessEnabled) out.push('unattendedAccess' as FeatureKey);
  return out;
}

export function unattendedDenialMessage(reason: UnattendedDenialReason): string {
  switch (reason) {
    case 'no_grant': return 'No unattended-access grant exists for this device.';
    case 'grant_revoked': return 'The unattended-access grant was revoked.';
    case 'grant_not_yet_valid': return 'The unattended-access grant is not active yet.';
    case 'grant_expired': return 'The unattended-access grant has expired.';
    case 'device_not_trusted': return 'Unattended access requires a trusted device.';
    case 'unattended_disabled_by_policy': return 'Unattended access is disabled by policy.';
    default: return 'Unattended access is not allowed.';
  }
}

/** Bridge to a PolicyDecision-style result for callers that want it. */
export function unattendedToPolicyDecision(d: UnattendedDecision): PolicyDecision {
  return d.allowed ? allow() : deny('host_not_accepted');
}

```

FILE: packages/shared/src/permissions/autoAccept.ts

```
import { evaluateUnattendedAccess, type UnattendedGrant } from './unattendedGrant';
import type { DeviceAccessPolicySnapshot } from './devicePolicy';
import type { SessionApprovalSnapshot } from './sessionApproval';

/**
 * Decides whether a session may auto-accept (skip the live consent prompt) under
 * an unattended grant. This is the ONLY sanctioned way to satisfy consent without
 * a human. Fail-closed: anything short of a valid grant on a trusted device with
 * unattended enabled falls back to normal (pending) consent.
 */
export interface AutoAcceptResult {
  /** True only when a valid unattended grant authorizes auto-accept. */
  autoAccepted: boolean;
  /** The approval snapshot to apply (approved when auto-accepted, else pending). */
  approval: SessionApprovalSnapshot;
}

export function evaluateAutoAccept(input: {
  policy: DeviceAccessPolicySnapshot | null;
  grant: UnattendedGrant | null;
  requestedByLabel?: string | null;
  now?: Date;
}): AutoAcceptResult {
  const decision = evaluateUnattendedAccess({ policy: input.policy, grant: input.grant, now: input.now });
  const at = (input.now ?? new Date()).toISOString();

  if (decision.allowed) {
    return {
      autoAccepted: true,
      approval: { status: 'approved', decidedAt: at, requestedByLabel: input.requestedByLabel ?? 'unattended grant' },
    };
  }
  // Fall back to normal consent: pending, requires a human.
  return {
    autoAccepted: false,
    approval: { status: 'pending', decidedAt: null, requestedByLabel: input.requestedByLabel ?? null },
  };
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval';
export * from './inputAction';
export * from './sessionAuditEvents';
export * from './reconnect';
export * from './fileTransferState';
export * from './clipboardSync';
export * from './deviceFingerprint';
export * from './trustDecision';
export * from './unattendedGrant'; // ADD
export * from './autoAccept';      // ADD
```

Part B — Desktop: unattended grant store

FILE: apps/desktop/src/renderer/src/services/unattendedGrantStore.ts

```
import {
  isGrantActive,
  evaluateUnattendedAccess,
  type DeviceAccessPolicySnapshot,
  type UnattendedDecision,
  type UnattendedGrant,
} from '@remotedesk/shared';

/**
 * Loads + caches the active unattended grant for a device and evaluates whether
 * unattended access may proceed. Fail-closed: a fetch failure yields no grant
 * (=> denied), and the store never fabricates a grant. Grants can be revoked
 * locally (immediately) in addition to server-side revocation.
 */
export interface GrantSource {
  /** Fetch the current grant for a device, or null. Throwing => treated as null. */
  fetchGrant: (deviceId: string) => Promise<UnattendedGrant | null>;
  /** Persist a revocation server-side. */
  revokeGrant: (grantId: string) => Promise<void>;
}

export class UnattendedGrantStore {
  private grant: UnattendedGrant | null = null;
  private listeners = new Set<(g: UnattendedGrant | null) => void>();

  constructor(private readonly source: GrantSource) {}

  /** Load the grant for a device. On any error => null (fail closed). */
  async load(deviceId: string): Promise<UnattendedGrant | null> {
    try {
      this.grant = await this.source.fetchGrant(deviceId);
    } catch {
      this.grant = null;
    }
    this.emit();
    return this.grant;
  }

  current(): UnattendedGrant | null {
    return this.grant;
  }

  /** Is there an active (in-window, non-revoked) grant right now? */
  hasActiveGrant(now: Date = new Date()): boolean {
    return !!this.grant && isGrantActive(this.grant, now);
  }

  /** Evaluate unattended access against a policy + the cached grant. */
  evaluate(policy: DeviceAccessPolicySnapshot | null, now: Date = new Date()): UnattendedDecision {
    return evaluateUnattendedAccess({ policy, grant: this.grant, now });
  }
}
```

```

/** Revoke immediately: clears locally + persists. Local clear is fail-safe. */
async revoke(): Promise<void> {
  const g = this.grant;
  this.grant = g ? { ...g, revokedAt: new Date().toISOString() } : null;
  this.emit();
  if (g) {
    try {
      await this.source.revokeGrant(g.grantId);
    } catch {
      // Local revocation already applied; server retry is the caller's concern.
    }
  }
}

subscribe(fn: (g: UnattendedGrant | null) => void): () => void {
  this.listeners.add(fn);
  return () => this.listeners.delete(fn);
}

private emit(): void {
  for (const fn of this.listeners) fn(this.grant);
}
}

```

Part C — Desktop: auto-accept controller

FILE: apps/desktop/src/renderer/src/services/autoAcceptController.ts

```

import {
  evaluateAutoAccept,
  makeAuditEvent,
  type DeviceAccessPolicySnapshot,
  type SessionApprovalSnapshot,
  type SessionAuditEvent,
  type UnattendedGrant,
} from '@remotedesk/shared';

/**
 * Bridges an unattended grant to the Pack 16 session-approval flow. At session
 * start it decides whether to auto-accept (under a valid grant) or fall back to a
 * pending human prompt. It emits a non-sensitive audit event either way and
 * re-evaluates on demand (e.g. after a reconnect re-check or grant revocation).
 */
export interface AutoAcceptDeps {
  /** Apply the resulting approval snapshot to the host approval state. */
  applyApproval: (approval: SessionApprovalSnapshot) => void;
  onAudit?: (e: SessionAuditEvent) => void;
}

export class AutoAcceptController {
  constructor(private readonly deps: AutoAcceptDeps) {}

  /** Returns true if the session auto-accepted under a grant. */
  evaluate(input: {
    policy: DeviceAccessPolicySnapshot | null;

```

```

    grant: UnattendedGrant | null;
    requestedByLabel?: string | null;
    now?: Date;
  }): boolean {
    const result = evaluateAutoAccept(input);
    this.deps.applyApproval(result.approval);
    this.deps.onAudit?.(
      makeAuditEvent(result.autoAccepted ? 'control.granted' : 'control.revoked', {
        mode: result.autoAccepted ? 'unattended_auto_accept' : 'attended_pending',
      }),
    );
    return result.autoAccepted;
  }
}

```

FILE: apps/desktop/src/renderer/src/services/sessionApprovalState.ts [MODIFY]

```

// ...existing Pack 16 imports...
import type { SessionApprovalSnapshot } from '@remotedesk/shared';

// ---- ADD: allow an externally-computed approval (from auto-accept) to be set ----
// This is the ONLY way approval becomes 'approved' without a human click, and it
// only ever receives an already-evaluated snapshot from AutoAcceptController
// (which itself requires a valid unattended grant). Fail-closed default unchanged.
//
// applyExternalApproval(approval: SessionApprovalSnapshot): void {
//   this.approval = approval;
//   this.emit();
// }
//
// NOTE: do NOT add any path that sets status:'approved' without going through
// AutoAcceptController -> evaluateAutoAccept -> a valid grant.

```

Part D — UI

FILE: apps/desktop/src/renderer/src/components/UnattendedGrantPanel.tsx

```

import React, { useEffect, useState } from 'react';
import { isGrantActive, type UnattendedGrant } from '@remotedesk/shared';
import type { UnattendedGrantStore } from '../services/unattendedGrantStore';

/**
 * Shows the current unattended-access grant + a revoke control. Read-only except
 * for Revoke, which kills auto-accept immediately (local + server).
 */
export function UnattendedGrantPanel({ store }: { store: UnattendedGrantStore }) {
  const [grant, setGrant] = useState<UnattendedGrant | null>(store.current());

  useEffect(() => store.subscribe(setGrant), [store]);

  if (!grant) {
    return <div className="unattended-panel unattended-panel--none">No unattended-access grant. Sessions require live approval.</div>;
  }

  const active = isGrantActive(grant);

```

```

return (
  <div className={`unattended-panel ${active ? 'unattended-panel--active' : 'unattended-panel--inactive'}`}>
    <div className="unattended-panel__row">
      <span>Unattended grant: {active ? 'active' : grant.revokedAt ? 'revoked' : 'inactive'}</span>
      <span className="unattended-panel__exp">expires {new Date(grant.expiresAt).toLocaleString()}</span>
    </div>
    <div className="unattended-panel__scope">scope: {grant.scope.join(', ') || 'none'}</div>
    {active && (
      <button type="button" className="btn btn--danger" onClick={() => void store.revoke()}>
        Revoke unattended access
      </button>
    )}
  </div>
);
}

```

FILE: apps/desktop/src/renderer/src/components/UnattendedBanner.tsx

```

import React from 'react';
import { unattendedDenialMessage, type UnattendedDecision } from '@remotedesk/shared';

/**
 * Banner shown while a session is running unattended (auto-accepted) OR explaining
 * why unattended access was denied and the session fell back to live approval.
 */
export function UnattendedBanner({ decision }: { decision: UnattendedDecision }) {
  if (decision.allowed) {
    return (
      <div className="unattended-banner unattended-banner--active" role="status">
        Unattended session active · scope: {decision.scope.join(', ') || 'view only'}
      </div>
    );
  }
  if (!decision.reason) return null;
  return (
    <div className="unattended-banner unattended-banner--denied" role="status">
      {unattendedDenialMessage(decision.reason)} Falling back to live approval.
    </div>
  );
}

```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```

// ...existing imports + Pack 15→20 additions...
import { UnattendedGrantPanel } from './UnattendedGrantPanel';
import { UnattendedBanner } from './UnattendedBanner';
import type { UnattendedDecision } from '@remotedesk/shared';

// ---- ADD: at session start, AFTER policy + trust are resolved (Pack 20), load
// the grant and run auto-accept BEFORE showing the consent prompt ----
//
// await unattendedGrantStore.load(deviceId);
// const unattendedDecision = unattendedGrantStore.evaluate(effectivePolicy);
// const autoAccepted = autoAcceptController.evaluate({
//   policy: effectivePolicy,           // policy with Pack 20 effective trustStatus
//   grant: unattendedGrantStore.current(),

```



```
// requestedByLabel: viewerLabel,
// });
// // If autoAccepted, sessionApprovalState is now 'approved' (no human needed).
// // If not, the Pack 16 SessionApprovalPrompt shows as normal.

// ---- ADD to render (host side): grant panel + unattended banner ----
// <UnattendedBanner decision={unattendedDecision} />
// <UnattendedGrantPanel store={unattendedGrantStore} />
//
// IMPORTANT: revoking the grant (panel) should re-evaluate -> approval falls back
// to pending; combine with Pack 18 reconnect re-check so a revoked grant cannot
// silently keep an unattended session interactive.
//
// All Pack 15→20 UI stays as-is. Screen view remains ungated.
```

Part E — Tests

FILE: tests/permissions/unattendedGrant.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  isGrantActive,
  evaluateUnattendedAccess,
  policyPermittedFeatures,
  type DeviceAccessPolicySnapshot,
  type UnattendedGrant,
} from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: true,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: false,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

function grant(over: Partial<UnattendedGrant> = {}): UnattendedGrant {
  return {
    grantId: 'g1', deviceId: 'dev-1', scope: ['remoteInput', 'clipboardSync', 'fileTransfer'] as any,
    notBefore: '2026-06-15T00:00:00.000Z', expiresAt: '2026-06-16T00:00:00.000Z',
    revokedAt: null, createdByLabel: 'admin', ...over,
  };
}

const NOW = new Date('2026-06-15T12:00:00.000Z');

describe('unattendedGrant', () => {
  it('isGrantActive respects window + revocation', () => {
    expect(isGrantActive(grant(), NOW)).toBe(true);
    expect(isGrantActive(grant({ revokedAt: '2026-06-15T11:00:00.000Z' }), NOW)).toBe(false);
    expect(isGrantActive(grant({ expiresAt: '2026-06-15T11:00:00.000Z' }), NOW)).toBe(false);
    expect(isGrantActive(grant({ notBefore: '2026-06-15T13:00:00.000Z' }), NOW)).toBe(false);
  });

  it('denies when device is untrusted', () => {
    const d = evaluateUnattendedAccess({ policy: policy({ trustStatus: 'untrusted' }), grant: grant(), now: NOW });
```

```

    expect(d.allowed).toBe(false);
    expect(d.reason).toBe('device_not_trusted');
  });

  it('denies when unattended disabled by policy', () => {
    const d = evaluateUnattendedAccess({ policy: policy({ unattendedAccessEnabled: false }), grant: grant(), now: NOW });
    expect(d.reason).toBe('unattended_disabled_by_policy');
  });

  it('denies when no grant / revoked / expired / not yet valid', () => {
    expect(evaluateUnattendedAccess({ policy: policy(), grant: null, now: NOW }).reason).toBe('no_grant');
    expect(evaluateUnattendedAccess({ policy: policy(), grant: grant({ revokedAt: '2026-06-15T11:00:00.000Z' }), now: NOW }).reason).toBe('grant_revoked');
    expect(evaluateUnattendedAccess({ policy: policy(), grant: grant({ expiresAt: '2026-06-15T11:00:00.000Z' }), now: NOW }).reason).toBe('grant_expired');
    expect(evaluateUnattendedAccess({ policy: policy(), grant: grant({ notBefore: '2026-06-15T13:00:00.000Z' }), now: NOW }).reason).toBe('grant_not_yet_valid');
  });

  it('clamps grant scope to policy-permitted features (never broader)', () => {
    // policy permits remoteInput + clipboard + unattended, NOT fileTransfer
    const d = evaluateUnattendedAccess({ policy: policy(), grant: grant(), now: NOW });
    expect(d.allowed).toBe(true);
    expect(d.scope).toContain('remoteInput');
    expect(d.scope).toContain('clipboardSync');
    expect(d.scope).not.toContain('fileTransfer'); // clamped out by policy
  });

  it('policyPermittedFeatures reflects the policy flags', () => {
    const f = policyPermittedFeatures(policy());
    expect(f).toContain('remoteInput');
    expect(f).not.toContain('fileTransfer');
  });
});

```

FILE: tests/permissions/autoAccept.test.ts

```

import { describe, it, expect } from 'vitest';
import { evaluateAutoAccept, type DeviceAccessPolicySnapshot, type UnattendedGrant } from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: true,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

function grant(over: Partial<UnattendedGrant> = {}): UnattendedGrant {
  return {
    grantId: 'g1', deviceId: 'dev-1', scope: ['remoteInput'] as any,
    notBefore: '2026-06-15T00:00:00.000Z', expiresAt: '2026-06-16T00:00:00.000Z',
    revokedAt: null, createdByLabel: 'admin', ...over,
  };
}

const NOW = new Date('2026-06-15T12:00:00.000Z');

```

```

describe('evaluateAutoAccept', () => {
  it('auto-accepts under a valid grant on a trusted device', () => {
    const r = evaluateAutoAccept({ policy: policy(), grant: grant(), now: NOW });
    expect(r.autoAccepted).toBe(true);
    expect(r.approval.status).toBe('approved');
  });

  it('falls back to pending when there is no grant', () => {
    const r = evaluateAutoAccept({ policy: policy(), grant: null, now: NOW });
    expect(r.autoAccepted).toBe(false);
    expect(r.approval.status).toBe('pending');
  });

  it('falls back to pending on an untrusted device even with a grant', () => {
    const r = evaluateAutoAccept({ policy: policy({ trustStatus: 'untrusted' }), grant: grant(), now: NOW });
    expect(r.autoAccepted).toBe(false);
    expect(r.approval.status).toBe('pending');
  });

  it('falls back to pending when the grant is expired', () => {
    const r = evaluateAutoAccept({ policy: policy(), grant: grant({ expiresAt: '2026-06-15T11:00:00.000Z' }), now: NOW });
    expect(r.autoAccepted).toBe(false);
    expect(r.approval.status).toBe('pending');
  });
});

```

FILE: tests/permissions/unattendedGrantStore.test.ts

```

import { describe, it, expect, vi } from 'vitest';
import { UnattendedGrantStore } from '../../apps/desktop/src/renderer/src/services/unattendedGrantStore';
import type { DeviceAccessPolicySnapshot, UnattendedGrant } from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: true,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

function grant(over: Partial<UnattendedGrant> = {}): UnattendedGrant {
  return {
    grantId: 'g1', deviceId: 'dev-1', scope: ['remoteInput'] as any,
    notBefore: '2026-06-15T00:00:00.000Z', expiresAt: '2026-06-16T00:00:00.000Z',
    revokedAt: null, createdByLabel: 'admin', ...over,
  };
}

const NOW = new Date('2026-06-15T12:00:00.000Z');

describe('UnattendedGrantStore', () => {
  it('loads a grant and reports it active', async () => {
    const store = new UnattendedGrantStore({ fetchGrant: vi.fn().mockResolvedValue(grant()), revokeGrant: vi.fn() });
    await store.load('dev-1');
    expect(store.hasActiveGrant(NOW)).toBe(true);
  });
});

```

```

    expect(store.evaluate(policy(), NOW).allowed).toBe(true);
  });

  it('fails closed when the fetch throws (no grant)', async () => {
    const store = new UnattendedGrantStore({ fetchGrant: vi.fn().mockRejectedValue(new Error('offline')), revokeGrant:
vi.fn() });
    await store.load('dev-1');
    expect(store.current()).toBeNull();
    expect(store.evaluate(policy(), NOW).reason).toBe('no_grant');
  });

  it('revoke clears locally immediately even if server revoke fails', async () => {
    const revokeGrant = vi.fn().mockRejectedValue(new Error('offline'));
    const store = new UnattendedGrantStore({ fetchGrant: vi.fn().mockResolvedValue(grant()), revokeGrant });
    await store.load('dev-1');
    await store.revoke();
    expect(store.hasActiveGrant(NOW)).toBe(false); // local revocation applied
    expect(revokeGrant).toHaveBeenCalled('g1');
  });
});

```

Part F — Reports

FILE: [generated-pack21-desktop-unattended-access-merge-summary.md](#)

Pack 21 (Desktop) — Unattended Access Pre-Authorization · Merge Summary

What this slice does

Continues Desktop Packs 15 → 20. Implements the last unused policy field, `unattendedAccessEnabled`, via time-boxed, scope-limited grants:

1. `UnattendedGrant` contract + evaluator: a session may auto-accept consent ONLY on a trusted device, with unattended enabled by policy, and a non-revoked, in-window grant. Grant scope is clamped to policy-permitted features (never broader).
2. `UnattendedGrantStore` (fail-closed load + immediate local revoke) and `AutoAcceptController` bridging grants into the Pack 16 approval flow.
3. UI: grant panel (with revoke) + unattended banner.

Everything funnels through the existing approval state, so a grant only substitutes for a human consent click — every Pack 15→20 gate still applies.

Files created (9)

- `packages/shared/src/permissions/unattendedGrant.ts`
- `packages/shared/src/permissions/autoAccept.ts`
- `apps/desktop/src/renderer/src/services/unattendedGrantStore.ts`
- `apps/desktop/src/renderer/src/services/autoAcceptController.ts`
- `apps/desktop/src/renderer/src/components/UnattendedGrantPanel.tsx`
- `apps/desktop/src/renderer/src/components/UnattendedBanner.tsx`
- `tests/permissions/unattendedGrant.test.ts`
- `tests/permissions/autoAccept.test.ts`
- `tests/permissions/unattendedGrantStore.test.ts`

Files modified (3)

- `packages/shared/src/permissions/index.ts` — export `unattendedGrant` + `autoAccept`.
- `apps/desktop/src/renderer/src/services/sessionApprovalState.ts` — `applyExternalApproval` (auto-accept only).
- `apps/desktop/src/renderer/src/components/RemoteSessionView.tsx` — grant panel + unattended banner + start-time auto-

accept.

Files intentionally NOT touched

- main/index.ts, preload/index.ts — no native executor / no bridge.
- webrtc.ts, permissionState.ts — Pack 15→17 gates unchanged.
- apps/api/**, apps/web/** — grant persistence assumed server-side; store falls back to no-grant if absent.

Remaining manual wiring risks

1. Provide GrantSource.fetchGrant/revokeGrant wired to your API (grant endpoint).
2. At session start (after Pack 20 trust resolution), load the grant + run AutoAcceptController BEFORE showing the consent prompt.
3. On revoke, re-evaluate so approval falls back to pending (pair with Pack 18 reconnect re-check).
4. Ensure applyExternalApproval is only ever called by AutoAcceptController.

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: trusted device + valid grant => session auto-accepts (no prompt), banner shows scope; expire/revoke the grant => next start needs live approval; untrusted device => never auto-accepts.

FILE: [generated-pack21-desktop-unattended-access-code-review.md](#)

Pack 21 (Desktop) — Code Review Notes

Strengths

- Auto-accept is the ONLY no-human consent path and requires a valid grant on a trusted device with unattended enabled — fail-closed otherwise.
- Grant scope is intersected with policy-permitted features, so a grant can never widen access beyond policy.
- Grants are time-boxed (notBefore/expiresAt) + revocable; local revoke applies instantly even if the server call fails.
- Reuses the Pack 16 approval state; no parallel consent path, so existing gates remain authoritative.
- Pure grant/auto-accept logic fully unit-tested; store tested with stubs.

Reviewer checklist

- [] GrantSource wired to the real grant API (fetch + revoke).
- [] AutoAcceptController.evaluate runs at session start before the consent prompt.
- [] applyExternalApproval is private to the auto-accept path (no other caller).
- [] Revocation triggers re-evaluation -> pending; reconnect re-check (Pack 18) honored.
- [] Unattended banner/scope reflect the clamped decision.scope, not the raw grant.

Safety

- No grant / expired / revoked / untrusted / unattended-disabled => no auto-accept (live consent required).
- Grant never broadens policy; scope is clamped.
- Native OS execution remains disabled/no-op (untouched).

FILE: [generated-pack21-desktop-unattended-access-manifest.json](#)

```
{
  "pack": "pack21-desktop-unattended-access",
  "title": "Unattended Access Pre-Authorization Grants + Auto-Accept Guards",
  "actualFileCount": 12,
  "filesCreated": [
    "packages/shared/src/permissions/unattendedGrant.ts",
    "packages/shared/src/permissions/autoAccept.ts",
    "apps/desktop/src/renderer/src/services/unattendedGrantStore.ts",
    "apps/desktop/src/renderer/src/services/autoAcceptController.ts",
    "apps/desktop/src/renderer/src/components/UnattendedGrantPanel.tsx",
    "apps/desktop/src/renderer/src/components/UnattendedBanner.tsx",
```

```

    "tests/permissions/unattendedGrant.test.ts",
    "tests/permissions/autoAccept.test.ts",
    "tests/permissions/unattendedGrantStore.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/sessionApprovalState.ts",
    "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/src/main/index.ts",
    "apps/desktop/src/preload/index.ts",
    "apps/desktop/src/renderer/src/services/webRTC.ts",
    "apps/desktop/src/renderer/src/services/permissionState.ts",
    "apps/api/**",
    "apps/web/**"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "react", "vitest"],
    "assumedFromProject": [
      "Desktop Pack 16 SessionApprovalState",
      "Desktop Pack 20 trust resolution (effective trustStatus in policy)",
      "Desktop Pack 17 sessionAuditHooks (optional audit)",
      "existing API endpoint for unattended grants (fetch + revoke)"
    ]
  },
  "knownLimitations": [
    "Grant creation UI is admin/web-side (out of scope); this slice consumes + revokes grants on desktop.",
    "Grant persistence assumed server-side; without it the store fails closed to no-grant (live approval).",
    "Auto-accept satisfies consent only; all other gates (input/clipboard/file/expiry) still apply.",
    "Native OS input execution remains disabled/no-op."
  ],
  "remainingManualWiringRisks": [
    "Wire GrantSource.fetchGrant/revokeGrant to the grant API.",
    "Run AutoAcceptController at session start before the consent prompt.",
    "Re-evaluate approval on grant revocation (+ Pack 18 reconnect re-check)."
  ],
  "estimatedCompletionAfterCarefulMerge": "72-78%"
}

```

Desktop Pack 21 done: 12 files (9 created, 3 modified). This implements the last untouched policy field, `unattendedAccessEnabled`, via time-boxed, scope-limited, revocable grants. A session can now auto-accept consent and run without a human at the host, but ONLY on a trusted device, with unattended enabled by policy, and a non-revoked, in-window grant.

The clean bit: auto-accept is the single sanctioned no-human consent path, and it funnels straight into Pack 16's existing approval state, so it merely substitutes for a consent click, every other gate (input, clipboard, file, expiry, reconnect re-check) still applies untouched. Grant scope is intersected with policy-permitted features, so a grant can never widen access beyond the policy, and local revoke applies instantly even if the server call fails. Everything short of a valid grant falls back to normal live approval.

The permission lifecycle is now end-to-end: 20 establishes trust → 21 pre-authorizes unattended use of it → 15 enforces → 16 consents (or 21 auto-accepts) → 17 funnels input → 18 re-checks on reconnect → 19 keeps transfers/clipboard safe. Next natural slice = consent receipt / session summary at disconnect, or the web-side admin UI to CREATE these grants. Bolo.